

Assignments for the course: Build a Data Mart in SQL (DLBDSPBDM01) - 1.1.2. Development phase/reflection phase

Alejandro Moral Aranda

August 16, 2025

Summary of Implementation

- ▶ The Airbnb database was designed based on an ER-model to capture users, properties, reservations, reviews, and supporting entities.
- ▶ Each entity is represented as a table, with foreign keys to ensure referential integrity.
- ▶ All SQL statements are well documented with inline comments for clarity.
- ▶ Each table contains at least 20 sample entries for testing and demonstration.
- ▶ Test cases are provided for each entity, with screenshots of query results.
- ▶ The implementation was tested using MySQL Command Line.

ER Diagram



Database Creation

```
-- Create the database if it doesn't exist and switch to it  
CREATE DATABASE IF NOT EXISTS 'airbnb'  
  /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE  
    utf8mb4_0900_ai_ci */; -- Use utf8mb4 for full Unicode  
    support  
USE 'airbnb'; -- Select the airbnb database
```

Drop Existing Tables

```
-- Disable foreign key checks for drops and creates (prevents
    errors during drop/create)
SET FOREIGN_KEY_CHECKS = 0;

-- Drop existing tables (in any order, due to disabled FK checks)
DROP TABLE IF EXISTS referral_rewards;           -- Drops
    referral_rewards if exists (ternary table for rewards)
DROP TABLE IF EXISTS dispute_resolution;         -- Drops
    dispute_resolution if exists (ternary table for disputes)
DROP TABLE IF EXISTS referral_programs;          -- Drops
    referral_programs if exists (referral tracking)
DROP TABLE IF EXISTS user_activity_logs;         -- Drops
    user_activity_logs if exists (user activity history)
DROP TABLE IF EXISTS support_tickets;            -- Drops
    support_tickets if exists (user support system)
DROP TABLE IF EXISTS cancellation_policies;      -- Drops
    cancellation_policies if exists (cancellation rules)
DROP TABLE IF EXISTS messages;                   -- Drops messages
    if exists (user messaging)
DROP TABLE IF EXISTS transactions;               -- Drops
    transactions if exists (payment records)
DROP TABLE IF EXISTS pricing_history;            -- Drops
```

Table Creation: user_profiles

```
-- 1. user_profiles: Stores all users' basic info (hosts, guests,
    admins)
CREATE TABLE user_profiles (
    user_id      INT          PRIMARY KEY,          -- Unique user
    identifier
    first_name   VARCHAR(50),  -- User's
    first name
    last_name    VARCHAR(50),  -- User's last
    name
    email        VARCHAR(100)  UNIQUE,             -- Unique
    email address
    phone        VARCHAR(20),  -- Contact
    phone number
    created_at   DATETIME,     -- Account
    creation timestamp
    user_role    VARCHAR(20)   DEFAULT 'guest'     -- Role: guest
    , host, admin, etc.
);
```

Table Creation: hosts

```
-- 2. hosts: Host-specific info, linked to user_profiles
CREATE TABLE hosts (
  host_id          INT PRIMARY KEY,          -- Unique host
  identifier
  user_id          INT,                      -- FK to
  user_profiles
  host_rating      DECIMAL(5,2),            -- Average
  host rating
  verification_status BOOLEAN,              -- Host
  verification status
  FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
);
```

Table Creation: guests

```
-- 3. guests: Guest-specific info, linked to user_profiles
CREATE TABLE guests (
    guest_id INT PRIMARY KEY,                -- Unique
    guest identifier
    user_id INT,                             -- FK to
    user_profiles
    FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
);
```

Table Creation: admin_users

```
-- 4. admin_users: Admin-specific info, linked to user_profiles
CREATE TABLE admin_users (
  admin_id      INT PRIMARY KEY,                -- Unique
  admin identifier
  user_id       INT,                            -- FK to
  user_profiles
  admin_level   INT,                            -- Admin
  privilege level
  privileges     TEXT,                          -- List of
  privileges
  reports_to    INT,                            -- FK to
  another admin (manager)
  FOREIGN KEY (user_id) REFERENCES user_profiles(user_id),
  FOREIGN KEY (reports_to) REFERENCES admin_users(admin_id)
);
```

Table Creation: `property_listings`

```
-- 5. property_listings: Main property info, linked to hosts
CREATE TABLE property_listings (
  property_id INT PRIMARY KEY,                -- Unique
  property identifier
  host_id INT,                                -- FK to hosts
  title VARCHAR(150),                          -- Listing
  title
  description TEXT,                            -- Listing
  description
  property_type VARCHAR(50),                  -- Type (
  apartment, house, etc.)
  room_type VARCHAR(50),                      -- Room type (
  entire, shared, etc.)
  address VARCHAR(255),                      -- Street
  address
  city VARCHAR(50),                          -- City
  state VARCHAR(50),                          -- State/
  region
  country VARCHAR(50),                      -- Country
  zipcode VARCHAR(10),                      -- Postal code
  created_at DATETIME,                      -- Listing
  creation timestamp
```

Table Creation: reservations

```
-- 6. reservations: Booking records, links guests and properties
CREATE TABLE reservations (
  reservation_id INT PRIMARY KEY,           -- Unique
    reservation identifier
  property_id    INT,                       -- FK to
    property_listings
  guest_id       INT,                       -- FK to
    guests
  check_in       DATE,                      -- Check-in
    date
  check_out      DATE,                      -- Check-out
    date
  status         VARCHAR(20),               -- Reservation
    status
  booked_at      DATETIME,                  -- Booking
    timestamp
  FOREIGN KEY (property_id) REFERENCES property_listings(
    property_id),
  FOREIGN KEY (guest_id)   REFERENCES guests(guest_id)
);
```

Table Creation: property_reviews

```
-- 7. property_reviews: Reviews for properties by guests
CREATE TABLE property_reviews (
    review_id    INT PRIMARY KEY,                -- Unique
    review_identifier
    property_id INT,                            -- FK to
    property_listings
    guest_id     INT,                          -- FK to guests
    rating       INT CHECK (rating BETWEEN 1 AND 5), -- Rating
    (1-5)
    comment      TEXT,                        -- Review text
    review_date  DATE,                       -- Date of
    review
    FOREIGN KEY (property_id) REFERENCES property_listings(
        property_id),
    FOREIGN KEY (guest_id) REFERENCES guests(guest_id)
);
```

Table Creation: host_reviews

```
-- 8. host_reviews: Reviews for hosts by guests
CREATE TABLE host_reviews (
  review_id    INT PRIMARY KEY,                -- Unique
  review_identifier
  host_id      INT,                            -- FK to hosts
  guest_id     INT,                            -- FK to guests
  rating       INT CHECK (rating BETWEEN 1 AND 5), -- Rating
  (1-5)
  comment      TEXT,                          -- Review text
  review_date  DATE,                          -- Date of
  review
  FOREIGN KEY (host_id) REFERENCES hosts(host_id),
  FOREIGN KEY (guest_id) REFERENCES guests(guest_id)
);
```

Table Creation: property_images

```
-- 9. property_images: Images for property listings
CREATE TABLE property_images (
    image_id      INT PRIMARY KEY,           -- Unique
    image_identifier
property_id INT,                           -- FK to
property_listings
image_url      VARCHAR(255),               -- Image URL
caption        VARCHAR(150),               -- Image
caption
uploaded_at    DATETIME,                   -- Upload
timestamp
FOREIGN KEY (property_id) REFERENCES property_listings(
property_id)
);
```

Table Creation: amenities

```
-- 10. amenities: List of possible amenities
CREATE TABLE amenities (
    amenity_id    INT PRIMARY KEY,                -- Unique
    amenity_identifier
    amenity_name  VARCHAR(50) UNIQUE,             -- Amenity name
    (unique)
    description   TEXT                             -- Amenity
    description
);
```

Table Creation: listing_amenities

```
-- 11. listing_amenities: Maps properties to amenities (many-to-many)
CREATE TABLE listing_amenities (
    property_id INT,                                -- FK to
    property_listings
    amenity_id INT,                                -- FK to
    amenities
    PRIMARY KEY (property_id, amenity_id),          -- Composite PK
    FOREIGN KEY (property_id) REFERENCES property_listings(
        property_id),
    FOREIGN KEY (amenity_id) REFERENCES amenities(amenity_id)
);
```

Table Creation: calendar_availability

```
-- 12. calendar_availability: Daily availability and price for
    each property
CREATE TABLE calendar_availability (
    property_id    INT,                                -- FK to
    property_listings
    available_date DATE,                                -- Date
    is_available   BOOLEAN,                             -- Availability
    status
    price          DECIMAL(8,2),                        -- Price for
    the date
    PRIMARY KEY (property_id, available_date),          -- Composite PK
    FOREIGN KEY (property_id) REFERENCES property_listings(
    property_id)
);
```

Table Creation: pricing_history

```
-- 13. pricing_history: Historical price changes for properties
CREATE TABLE pricing_history (
    pricing_id INT PRIMARY KEY,                -- Unique
    pricing_record
    property_id INT,                          -- FK to
    property_listings
    price DECIMAL(8,2),                       -- Price
    start_date DATE,                         -- Price start
    date
    end_date DATE,                           -- Price end
    date
    FOREIGN KEY (property_id) REFERENCES property_listings(
    property_id)
);
```

Table Creation: transactions

```
-- 14. transactions: Payment transactions for reservations
CREATE TABLE transactions (
    transaction_id    INT PRIMARY KEY,           -- Unique
    transaction identifier
    reservation_id    INT,                       -- FK to
    reservations
    transaction_date  DATETIME,                  -- Transaction
    timestamp
    amount            DECIMAL(10,2),             -- Amount paid
    payment_method    VARCHAR(50),              -- Payment
    method
    status            VARCHAR(20),               -- Transaction
    status
    FOREIGN KEY (reservation_id) REFERENCES reservations(
        reservation_id)
);
```

Table Creation: messages

```
-- 15. messages: Messaging between users (per reservation)
CREATE TABLE messages (
  message_id      INT PRIMARY KEY,                -- Unique
  message_identifier
  reservation_id  INT,                            -- FK to
  reservations
  sender_id       INT,                            -- FK to
  user_profiles (sender)
  receiver_id     INT,                            -- FK to
  user_profiles (receiver)
  content          TEXT,                          -- Message
  content
  sent_at         DATETIME,                       -- Sent
  timestamp
  FOREIGN KEY (reservation_id) REFERENCES reservations(
    reservation_id),
  FOREIGN KEY (sender_id)      REFERENCES user_profiles(user_id
  ),
  FOREIGN KEY (receiver_id)    REFERENCES user_profiles(user_id
  )
);
```

Table Creation: `property_rules`

```
-- 16. property_rules: Rules for each property
CREATE TABLE property_rules (
    rule_id          INT PRIMARY KEY,          -- Unique rule
        identifier
    property_id      INT,                      -- FK to
        property_listings
    rule_description TEXT,                    -- Rule
        description
    FOREIGN KEY (property_id) REFERENCES property_listings(
        property_id)
);
```

Table Creation: cancellation_policies

```
-- 17. cancellation_policies: Cancellation/refund policies
CREATE TABLE cancellation_policies (
  policy_id          INT PRIMARY KEY,          -- Unique
  policy_identifier
  policy_name        VARCHAR(50),              -- Policy name
  description         TEXT,                    -- Policy
  description
  refund_percentage DECIMAL(5,2)              -- Refund
  percent (0-100)
);
```

Table Creation: support_tickets

```
-- 18. support_tickets: User support requests
CREATE TABLE support_tickets (
    ticket_id    INT PRIMARY KEY,                -- Unique
    ticket identifier
    user_id      INT,                            -- FK to
    user_profiles
    subject      VARCHAR(150),                  -- Ticket
    subject
    description TEXT,                           -- Ticket
    details
    status       VARCHAR(20),                   -- Ticket
    status
    created_at   DATETIME,                      -- Ticket
    creation time
    resolved_at  DATETIME NULL,                 -- Resolution
    time (nullable)
    FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
);
```

Table Creation: user_activity_logs

```
-- 19. user_activity_logs: Tracks user actions/events
CREATE TABLE user_activity_logs (
  log_id          INT PRIMARY KEY,           -- Unique log
  identifier
  user_id         INT,                       -- FK to
  user_profiles
  activity_type   VARCHAR(50),              -- Type of
  activity
  activity_timestamp DATETIME,              -- When it
  happened
  description     TEXT,                     -- Details
  FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
);
```

Table Creation: referral_programs

```
-- 20. referral_programs: Tracks user referrals
CREATE TABLE referral_programs (
    referral_id          INT PRIMARY KEY,           -- Unique
    referral_identifier
    referrer_user_id    INT,                       -- FK to
    user_profiles (referrer)
    referred_user_id    INT,                       -- FK to
    user_profiles (referred)
    referral_code        VARCHAR(50),              -- Referral
    code
    created_at          DATETIME,                  -- Referral
    creation time
    status              VARCHAR(20),               -- Referral
    status
    FOREIGN KEY (referrer_user_id) REFERENCES user_profiles(
    user_id),
    FOREIGN KEY (referred_user_id) REFERENCES user_profiles(
    user_id)
);
```

Table Creation: dispute_resolution

```
-- 21. dispute_resolution (ternary): Resolves disputes involving
    tickets, admins, reservations
CREATE TABLE dispute_resolution (
    resolution_id      INT PRIMARY KEY,           -- Unique
    resolution_identifier
    ticket_id          INT,                       -- FK to
    support_tickets
    admin_id           INT,                       -- FK to
    admin_users
    reservation_id     INT,                       -- FK to
    reservations
    resolution_date     DATETIME,                 -- Date
    resolved
    resolution_details TEXT,                      -- Details of
    resolution
    FOREIGN KEY (ticket_id) REFERENCES support_tickets(
    ticket_id),
    FOREIGN KEY (admin_id) REFERENCES admin_users(admin_id)
    ,
    FOREIGN KEY (reservation_id) REFERENCES reservations(
    reservation_id)
);
```

Table Creation: referral_rewards

```
-- 22. referral_rewards (ternary): Rewards for referrals, linked
    to transactions and admins
CREATE TABLE referral_rewards (
    reward_id      INT PRIMARY KEY,                -- Unique
    reward_identifier
    referral_id    INT,                            -- FK to
    referral_programs
    transaction_id INT,                            -- FK to
    transactions
    admin_id       INT,                            -- FK to
    admin_users
    reward_amount  DECIMAL(10,2),                 -- Reward
    amount
    reward_date    DATETIME,                       -- Date
    rewarded
    FOREIGN KEY (referral_id) REFERENCES referral_programs(
    referral_id),
    FOREIGN KEY (transaction_id) REFERENCES transactions(
    transaction_id),
    FOREIGN KEY (admin_id) REFERENCES admin_users(admin_id)
);
```

Test Case 1: Volume Check

```
-- Every table should report 20 rows
SELECT 'user_profiles' AS table_name, COUNT(*) AS row_count FROM
    user_profiles
UNION ALL SELECT 'hosts', COUNT(*) FROM hosts
UNION ALL SELECT 'guests', COUNT(*) FROM guests
UNION ALL SELECT 'admin_users', COUNT(*) FROM admin_users
UNION ALL SELECT 'property_listings', COUNT(*) FROM
    property_listings
UNION ALL SELECT 'reservations', COUNT(*) FROM reservations
UNION ALL SELECT 'property_reviews', COUNT(*) FROM
    property_reviews
UNION ALL SELECT 'host_reviews', COUNT(*) FROM host_reviews
UNION ALL SELECT 'property_images', COUNT(*) FROM property_images
UNION ALL SELECT 'amenities', COUNT(*) FROM amenities
UNION ALL SELECT 'listing_amenities', COUNT(*) FROM
    listing_amenities
UNION ALL SELECT 'calendar_availability', COUNT(*) FROM
    calendar_availability
UNION ALL SELECT 'pricing_history', COUNT(*) FROM pricing_history
UNION ALL SELECT 'transactions', COUNT(*) FROM transactions
UNION ALL SELECT 'messages', COUNT(*) FROM messages
UNION ALL SELECT 'property_rules', COUNT(*) FROM property_rules
```

Test Case 1: Volume Check

- ▶ **Purpose:** Verify that every core table in the data mart is sufficiently populated with test data (at least 20 rows per table).
- ▶ **Explanation:** Sufficient row volume is essential for meaningful testing, analytics, and performance checks. This query quickly reveals if any table is underpopulated, which may indicate issues with data loading or test setup.
- ▶ **What Constitutes a Pass:** All tables should show a row count of 20 or more. If any table reports fewer rows, it signals that further data generation or correction is needed.

Test Case 1: Result

```
mysql> SOURCE c:/Users/pc/Project-Building-a-Data-Mart-in-SQL/test.sql;
Database changed
+-----+-----+
| table_name | row_count |
+-----+-----+
| user_profiles | 30 |
| hosts | 20 |
| guests | 20 |
| admin_users | 20 |
| property_listings | 20 |
| reservations | 20 |
| property_reviews | 20 |
| host_reviews | 20 |
| property_images | 20 |
| amenities | 20 |
| listing_amenities | 40 |
| calendar_availability | 48 |
| pricing_history | 20 |
| transactions | 20 |
| messages | 20 |
| property_rules | 20 |
| cancellation_policies | 20 |
| support_tickets | 20 |
| user_activity_logs | 20 |
| referral_programs | 20 |
| dispute_resolution | 20 |
| referral_rewards | 20 |
+-----+-----+
22 rows in set (0.00 sec)

+-----+-----+
| IF(MIN(row_count) >= 20, 'Volume check: OK', 'Volume check: FAIL') |
+-----+-----+
| Volume check: OK |
+-----+-----+
```

Test Case 2: Positive Join Test

```
-- Booking guest name property payment
SELECT r.reservation_id,
       CONCAT(up.first_name,' ',up.last_name) AS guest,
       p.title                               AS property,
       t.amount                               AS paid
FROM   reservations      AS r
JOIN   guests            AS g   ON g.guest_id      = r.guest_id
JOIN   user_profiles     AS up  ON up.user_id      = g.user_id
JOIN   property_listings AS p   ON p.property_id   = r.
      property_id
JOIN   transactions      AS t   ON t.reservation_id = r.
      reservation_id
LIMIT 5;
```

Test Case 2: Positive Join Test

- ▶ **Purpose:** Confirm that logical relationships between bookings, guests, properties, and transactions are correctly established and navigable via joins.
- ▶ **Explanation:** This test simulates a real-world use case: retrieving booking details along with guest identity, property info, and payment amount. Successful joins across these tables demonstrate correct foreign key usage and data integrity.
- ▶ **What Constitutes a Pass:** The query returns rows with all fields populated (reservation ID, guest name, property, paid amount), showing that the join path is valid and works as expected.

Test Case 2: Result

```
+-----+-----+-----+-----+
| reservation_id | guest          | property | paid  |
+-----+-----+-----+-----+
|              1 | Juan Pérez    | Listing 1 | 500.00 |
|              2 | Beatriz Sánchez | Listing 2 | 450.00 |
|              3 | Carlos Díaz   | Listing 3 | 300.00 |
|              4 | Diego Romero  | Listing 4 | 700.00 |
|              5 | Elena García  | Listing 5 | 600.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

+-----+
| IF(EXISTS (
|   SELECT 1
|   FROM    reservations r
|   JOIN    guests g ON g.guest_id = r.guest_id
|   JOIN    user_profiles up ON up.user_id = g.user_id
|   JOIN    property_listings p ON p.property_id = r.property_id
|   JOIN    transactions t ON t.reservation_id = r.reservation_id
| )
+-----+
| Positive join test: OK
|
```

Test Case 3: Ternary Join Test

```
-- dispute_resolution links its three parents
SELECT dr.resolution_id,
       st.subject          AS ticket_subject,
       au.admin_level,
       res.check_in
FROM   dispute_resolution AS dr
JOIN   support_tickets    AS st  ON st.ticket_id      = dr.
      ticket_id
JOIN   admin_users        AS au  ON au.admin_id       = dr.
      admin_id
JOIN   reservations       AS res ON res.reservation_id = dr.
      reservation_id
LIMIT 5;
```

Test Case 3: Ternary Join Test

- ▶ **Purpose:** Validate the integrity of complex (ternary) relationships, ensuring the dispute resolution table correctly references all three parent entities.
- ▶ **Explanation:** By joining dispute-resolution with its referenced support tickets, admins, and reservations, this query checks that multi-way (ternary) foreign key constraints are respected and that join queries for such relationships function as intended.
- ▶ **What Constitutes a Pass:** The query produces rows with a resolution ID, ticket subject, admin level, and check-in date, confirming that all three-way relationships are present and correct.

Test Case 3: Result

```
+-----+-----+-----+-----+
| resolution_id | ticket_subject | admin_level | check_in |
+-----+-----+-----+-----+
|          1   | Refund         |          2   | 2025-01-05 |
|          2   | Calendar bug   |          1   | 2025-01-08 |
|          3   | Spam           |          1   | 2025-01-12 |
|          4   | Payout delay   |          1   | 2025-01-15 |
|          5   | Security       |          1   | 2025-01-20 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
+-----+
| IF(EXISTS (
  SELECT 1
  FROM    dispute_resolution dr
  JOIN    support_tickets st ON st.ticket_id = dr.ticket_id
  JOIN    admin_users au ON au.admin_id = dr.admin_id
  JOIN    reservations res ON res.reservation_id = dr.reservation_id
), 'Ternary join test |
+-----+
| Ternary join test: OK
|
```

Test Case 4a: Negative FK Test

```
-- Should raise ERROR 1452, then roll back
START TRANSACTION;
INSERT INTO reservations
(reservation_id, property_id, guest_id,
 check_in, check_out, status, booked_at)
VALUES (9999, 1, 9999, '2025-12-01', '2025-12-05', 'pending', NOW
      ());
ROLLBACK;
```

Test Case 4a: Negative FK Test

- ▶ **Purpose:** Test the enforcement of foreign key (FK) constraints by attempting to insert an invalid reference.
- ▶ **Explanation:** This test tries to create a reservation for a non-existent guest (guest-id 9999). The FK constraint on guest-id should prevent this, causing the database to throw an error and reject the insert. The transaction is rolled back to avoid leaving invalid data.
- ▶ **What Constitutes a Pass:** The operation fails with a foreign key error (e.g., ERROR 1452), demonstrating that referential integrity is enforced.

Test Case 4b: Negative CHECK Test

```
-- Rating outside 15 should raise ERROR 3819
START TRANSACTION;
INSERT INTO property_reviews
(review_id, property_id, guest_id, rating, comment, review_date)
VALUES (9999, 1, 1, 7,
       'this should fail rating > 5', CURDATE());
ROLLBACK;
```

Test Case 4b: Negative CHECK Test

- ▶ **Purpose:** Ensure that CHECK constraints are functioning by attempting to insert an out-of-range value.
- ▶ **Explanation:** The rating column in property-reviews is restricted to values between 1 and 5. This test attempts to insert a rating of 7, which should be blocked by the CHECK constraint. The transaction is rolled back to avoid corrupting the data.
- ▶ **What Constitutes a Pass:** The operation fails with a CHECK constraint error (e.g., ERROR 3819), confirming that the database enforces the defined data validity rules.

Test Case 4: Result

```
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('airbnb`.`reservations`, CONSTRAINT `reservations_ibfk_2` FOREIGN KEY (guest_id) REFERENCES `guests` (`guest_id`))
Query OK, 0 rows affected (0.01 sec)

+-----+
| result |
+-----+
| Negative FK test: OK if previous error 1452 was shown |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 3819 (HY000): Check constraint 'property_reviews_chk_1' is violated.
Query OK, 0 rows affected (0.00 sec)

+-----+
| result |
+-----+
| Negative CHECK test: OK if previous error 3819 was shown |
+-----+
1 row in set (0.00 sec)
```

Conclusion

- ▶ The Airbnb database schema is fully implemented and tested.
- ▶ All entities and relationships from the ER model are represented.
- ▶ Each table is populated with at least 20 entries for robust testing.
- ▶ Test cases confirm the correctness of the schema and data.
- ▶ The design is ready for further development and integration.